VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT

on

OPERATING SYSTEMS

Submitted by

ADITI KAMATH A

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "OPERATING SYSTEMS – 23CS4PCOPS" carried out by ADITI KAMATH A (1BM24CS016), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

SEEMA PATIL	Dr. Kavitha Sooda
Assistant Professor	Professor and Head
Department of CSE	Department of CSE
BMSCE, Bengaluru	BMSCE, Bengaluru

Index Sheet

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program1a

Question: Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

- FCFS
- SJF (pre-emptive & Non-preemptive)

Soln: =>FCFS

```
#include<stdio.h>
struct Processes
{
   int ID,AT,BT,CT,TAT,WT;
};
int main()
{
   int n;
   printf("Number of processes:");
   scanf("%d",&n);
   struct Processes p[n];
   for(int i=0;i<n;i++)
   {
      p[i].ID=i+1;
      printf("Arrival time of P%d:",p[i].ID);
      scanf("%d",&p[i].AT);
      printf("Burst time of P%d:",p[i].ID);
      scanf("%d",&p[i].BT);
   }
   for(int i=0;i<n;i++)
   {
      for(int j=0;j<n-i-1;j++)
      {
         if(p[j].AT>p[j+1].AT)
         {
            struct Processes temp=p[j];
            p[j]=p[j+1];
            p[j+1]=temp;
         }
      }
   }
   int time_passed=0,sumTAT=0,sumWT=0;
   for (int i=0;i<n;i++)
   {
      if(time_passed<p[i].AT)
      {
         time_passed=p[i].AT;
      }
      p[i].CT=time_passed+p[i].BT;
      p[i].TAT=p[i].CT-p[i].AT;
      p[i].WT=p[i].TAT-p[i].BT;
      time_passed=p[i].CT;
```

```
        sumTAT+=p[i].TAT;
        sumWT+=p[i].WT;
    }
    printf("ID\tAT\tBT\tCT\tTAT\tWT");
    for (int i=0;i<n;i++)
    {
        printf("\n%d\t%d\t%d\t%d\t%d\t%d",p[i].ID,p[i].AT,p[i].BT,p[i].CT,p[i].TAT,p[i].WT);
    }
    printf("\nAverage TAT:%.2f\n",(float)sumTAT/n);
    printf("Average WT:%.2f",(float)sumWT/n);
}
```

```
C:\Users\BMSCECSE-L4>CD "C:\Users\BMSCECSE-L4\Desktop\1BM24CS016"

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>FCFS
Number of processes:4
Arrival time of P1:0
Burst time of P1:2
Arrival time of P2:1
Burst time of P2:2
Arrival time of P3:5
Burst time of P3:3
Arrival time of P4:6
Burst time of P4:4
ID      AT      BT      CT      TAT     WT
1       0       2       2       2       0
2       1       2       4       3       1
3       5       3       8       3       0
4       6       4       12      6       2
Average TAT:3.50
Average WT:0.75
C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>
```

SOLN: =>SJF(Preemptive) -SRTF

```
#include <stdio.h>

struct Process {
    int id, at, bt, ct, tat, wt, rt, rem;
};

int main() {
    int n;
    printf("Enter no. of process: ");
    scanf("%d", &n);

    struct Process p[n];
    for(int i = 0; i < n; i++) {
```

```
    p[i].id = i + 1;
    printf("Arrival Time of P%d: ", p[i].id);
    scanf("%d", &p[i].at);
    printf("Burst Time of P%d: ", p[i].id);
    scanf("%d", &p[i].bt);
    p[i].rem = p[i].bt;
    p[i].rt = -1;
}

int completed = 0, time = 0, sumTAT = 0, sumWT = 0,sumRT=0;

while (completed != n) {
    int idx = -1, minRem = 1e9;

    for(int i = 0; i < n; i++) {
        if (p[i].at <= time && p[i].rem > 0) {
            if (p[i].rem < minRem) {
                minRem = p[i].rem;
                idx = i;
            }
        }
    }

    if (idx != -1) {
        if (p[idx].rt == -1) {
            p[idx].rt = time - p[idx].at;
            sumRT += p[idx].rt;
        }
        p[idx].rem--; time++;
        if (p[idx].rem == 0) {
            p[idx].ct = time;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;

            sumTAT += p[idx].tat;
            sumWT += p[idx].wt;
            sumRT += p[idx].rt; completed++;
        }
    }
    else { time++; }
}
printf("ID\tAT\tBT\tCT\tTAT\tWT");
for (int i=0;i<n;i++)
{
    printf("\n%d\t%d\t%d\t%d\t%d\t%d",p[i].id,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
}
printf("\nAvg TAT: %.2f\n", (float)sumTAT/n);
printf("Avg WT: %.2f\n", (float)sumWT/n);
printf("Avg RT: %.2f\n", (float)sumRT/n);
return 0;
```

}

```
C:\Users\BMSCECSE-L4>CD "C:\Users\BMSCECSE-L4\Desktop\1BM24CS016"

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>SRTF
Enter no. of process: 5
Arrival Time of P1: 2
Burst Time of P1: 1
Arrival Time of P2: 1
Burst Time of P2: 5
Arrival Time of P3: 4
Burst Time of P3: 1
Arrival Time of P4: 0
Burst Time of P4: 6
Arrival Time of P5: 2
Burst Time of P5: 3
ID      AT      BT      CT      TAT     WT
1       2       1       3       1       0
2       1       5       11      10      5
3       4       1       5       1       0
4       0       6       16      16      10
5       2       3       7       5       2
Avg TAT: 6.60
Avg WT: 3.40
Avg RT: 0.40

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>
```

SOLN: => SJF Non preemptive

```
#include <stdio.h>

struct Process {
    int id, at, bt, ct, tat, wt, rt;
};

int main() {
    int n;
    printf("Enter no. of processes: "); scanf("%d", &n);

    struct Process p[n];
    for(int i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("Arrival Time of P%d: ", p[i].id);
        scanf("%d", &p[i].at);
        printf("Burst Time of P%d: ", p[i].id);
        scanf("%d", &p[i].bt);
    }
```

```c
    int completed = 0, time = 0, sumTAT = 0, sumWT = 0;
    int isComplete[n];
    for(int i = 0; i < n; i++)
        isComplete[i] = 0;

    while (completed != n) {
        int idx = -1, minBT = 1e9;

        for(int i = 0; i < n; i++) {
            if (p[i].at <= time && !isComplete[i]) {
                if (p[i].bt < minBT) {
                    minBT = p[i].bt;
                    idx = i;
                }
            }
        }

        if (idx != -1) {
            p[idx].ct = time + p[idx].bt;
            p[idx].tat = p[idx].ct - p[idx].at;
            p[idx].wt = p[idx].tat - p[idx].bt;

            sumTAT += p[idx].tat;
            sumWT += p[idx].wt;

            time += p[idx].bt;
            isComplete[idx] = 1;
            completed++;
        } else { time++; }
    }
    printf("ID\tAT\tBT\tCT\tTAT\tWT");
    for (int i=0;i<n;i++)
    {
        printf("\n%d\t%d\t%d\t%d\t%d\t%d",p[i].id,p[i].at,p[i].bt,p[i].ct,p[i].tat,p[i].wt);
    }

    printf("\nAverage TAT: %.2f\n", (float)sumTAT/n);
    printf("Average WT: %.2f\n", (float)sumWT/n);

    return 0;
}
```

```
C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>SJF_nonpreemptive
Enter no. of processes: 4
Arrival Time of P1: 0
Burst Time of P1: 6
Arrival Time of P2: 0
Burst Time of P2: 8
Arrival Time of P3: 0
Burst Time of P3: 7
Arrival Time of P4: 0
Burst Time of P4: 3
ID      AT      BT      CT      TAT     WT
1       0       6       9       9       3
2       0       8       24      24      16
3       0       7       16      16      9
4       0       3       3       3       0
Average TAT: 13.00
Average WT: 7.00

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>
```

PROGRAM 1b

Question : Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

→ Priority (pre-emptive & Non-pre-emptive)

→Round Robin (Experiment with different quantum sizes for RR algorithm)

SOLN: Priority Pre-emptive

```
#include <stdio.h>

struct Process {
    int pid, at, bt, rt, priority, ct, wt, tat;
};

int main() {
    int n, sumWT = 0, sumTAT = 0;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process p[n];

    for(int i=0; i<n; i++) {
        p[i].pid = i+1;
        printf("Enter AT, BT and Priority for P%d: ", i+1);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].priority);
```

```
        p[i].rt = p[i].bt;
    }

    int completed = 0, time = 0, min_pri;
    int shortest = -1;
    int check = 0;

    while(completed != n) {
        min_pri = 10000;   // large initial value
        shortest = -1;
        check = 0;

        for(int i=0; i<n; i++) {
            if((p[i].at <= time) && (p[i].rt > 0) && (p[i].priority <
min_pri)) {
                min_pri = p[i].priority;
                shortest = i;
                check = 1;
            }
        }

        if(check == 0) {
            time++;
            continue;
        }

        p[shortest].rt--;
        time++;

        if(p[shortest].rt == 0) {
            completed++;
            p[shortest].ct = time;
            p[shortest].tat = p[shortest].ct - p[shortest].at;
            p[shortest].wt = p[shortest].tat - p[shortest].bt;

            sumWT += p[shortest].wt;
            sumTAT += p[shortest].tat;
        }
    }

    printf("\nPID\tAT\tBT\tPri\tCT\tWT\tTAT\n");
    for(int i=0; i<n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].at, p[i].bt, p[i].priority,
            p[i].ct, p[i].wt, p[i].tat);
    }
    printf("Average WT: %.2f\n",(float)sumWT/n);
    printf("Average TAT: %.2f\n",(float)sumTAT/n);

    return 0;
```

```
}
```

```
C:\Users\BMSCECSE-L4>CD C:\Users\BMSCECSE-L4\Desktop\1BM24CS016

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>Pri_preempt
Enter number of processes: 5
Enter AT, BT and Priority for P1: 0
3
3
Enter AT, BT and Priority for P2: 1
4
2
Enter AT, BT and Priority for P3: 2
6
4
Enter AT, BT and Priority for P4: 3
4
6
Enter AT, BT and Priority for P5: 5
2
10

PID     AT      BT      Pri     CT      WT      TAT
1       0       3       3       7       4       7
2       1       4       2       5       0       4
3       2       6       4       13      5       11
4       3       4       6       17      10      14
5       5       2       10      19      12      14
Average WT: 6.20
Average TAT: 10.00

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>
```

SOLN :Priority Non premetive

```
#include <stdio.h>

struct Process {
    int pid, at, bt, priority, ct, wt, tat, is_completed;
};

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    struct Process p[n];

    for(int i=0; i<n; i++) {
        p[i].pid = i+1;
        printf("Enter AT, BT and Priority for P%d: ", i+1);
        scanf("%d %d %d", &p[i].at, &p[i].bt, &p[i].priority);
        p[i].is_completed = 0;
```

```c
    }

    int completed = 0, time = 0, min_pri;
    int selected = -1;

    while(completed != n) {
        min_pri = 10000;
        selected = -1;

        for(int i=0; i<n; i++) {
            if((p[i].at <= time) && (p[i].is_completed == 0) && (p[i].priority < min_pri)) {
                min_pri = p[i].priority;
                selected = i;
            }
        }

        if(selected == -1) {
            time++;
            continue;
        }

        time += p[selected].bt;
        p[selected].ct = time;
        p[selected].tat = p[selected].ct - p[selected].at;
        p[selected].wt = p[selected].tat - p[selected].bt;
        p[selected].is_completed = 1;
        completed++;
    }

    float avg_tat = 0, avg_wt = 0;
    printf("\nPID\tAT\tBT\tPri\tCT\tWT\tTAT\n");
    for(int i=0; i<n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
               p[i].pid, p[i].at, p[i].bt, p[i].priority,
               p[i].ct, p[i].wt, p[i].tat);
        avg_tat += p[i].tat;
        avg_wt += p[i].wt;
    }

    avg_tat /= n;
    avg_wt /= n;

    printf("\nAverage TAT = %.2f", avg_tat);
    printf("\nAverage WT = %.2f\n", avg_wt);

    return 0;
}
```

```
C:\Users\BMSCECSE-L4>cd "C:\Users\BMSCECSE-L4\Desktop\1BM24CS016"

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>Pri_nonpreempt
Enter number of processes: 5
Enter AT, BT and Priority for P1: 0
3
5
Enter AT, BT and Priority for P2: 2
2
3
Enter AT, BT and Priority for P3: 3
5
2
Enter AT, BT and Priority for P4: 4
4
4
Enter AT, BT and Priority for P5: 6
1
1

PID     AT      BT      Pri     CT      WT      TAT
1       0       3       5       3       0       3
2       2       2       3       11      7       9
3       3       5       2       8       0       5
4       4       4       4       15      7       11
5       6       1       1       9       2       3

Average TAT = 6.20
Average WT = 3.20

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>
```

SOLN : Round Robin

```
#include <stdio.h>

struct Process {
    int pid, at, bt, rt, ct, wt, tat;
};


void sortArrival(struct Process p[], int n) {
    struct Process temp;
    for(int i=0; i<n-1; i++) {
        for(int j=0; j<n-i-1; j++) {
            if(p[j].at > p[j+1].at) {
                temp = p[j]; p[j] = p[j+1]; p[j+1] = temp;
            }
        }
    }
}

int main() {
    int n, quantum, completed = 0, curr_time = 0;
    float total_wt = 0, total_tat = 0;
    int queue[100], front = 0, rear = 0;
    int mark[100] = {0};
```

```
printf("Enter number of processes: ");
scanf("%d", &n);
struct Process p[n];

for(int i=0; i<n; i++) {
    p[i].pid = i + 1;
    printf("Enter Arrival Time and Burst Time for P%d: ", p[i].pid);
    scanf("%d %d", &p[i].at, &p[i].bt);
    p[i].rt = p[i].bt;
}

printf("Enter Time Quantum: ");
scanf("%d", &quantum);

sortArrival(p, n);


curr_time = p[0].at;
queue[rear++] = 0;
mark[0] = 1;

while(completed != n) {
    int idx = queue[front++];


    int slice = (p[idx].rt > quantum) ? quantum : p[idx].rt;
    p[idx].rt -= slice;
    curr_time += slice;


    for(int i = 0; i < n; i++) {
        if(p[i].at <= curr_time && mark[i] == 0 && p[i].rt > 0) {
            queue[rear++] = i;
            mark[i] = 1;
        }
    }

    if(p[idx].rt > 0) {
        queue[rear++] = idx;
    } else {

        p[idx].ct = curr_time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        total_wt += p[idx].wt;
        total_tat += p[idx].tat;
        completed++;
    }
```

```
        if(front == rear && completed < n) {
            for(int i = 0; i < n; i++) {
                if(mark[i] == 0) {
                    curr_time = p[i].at;
                    queue[rear++] = i;
                    mark[i] = 1;
                    break;
                }
            }
        }
    }


    printf("\nPID\tAT\tBT\tCT\tTAT\tWT\n");
    for(int i=0; i<n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage Waiting Time: %.2f", total_wt / n);
    printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);

    return 0;
}
```

```
C:\Users\BMSCECSE-L4>cd C:\Users\BMSCECSE-L4\Desktop\1BM24CS016

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>Round_robin
Enter number of processes: 5
Enter Arrival Time and Burst Time for P1: 0
5
Enter Arrival Time and Burst Time for P2: 1
3
Enter Arrival Time and Burst Time for P3: 2
1
Enter Arrival Time and Burst Time for P4: 3
2
Enter Arrival Time and Burst Time for P5: 4
3
Enter Time Quantum: 2

PID     AT      BT      CT      TAT     WT
P1      0       5       13      13      8
P2      1       3       12      11      8
P3      2       1       5       3       2
P4      3       2       9       6       4
P5      4       3       14      10      7

Average Waiting Time: 5.80
Average Turnaround Time: 8.60

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>
```

PROGRAM 2

Question : Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

SOLN :

```
#include <stdio.h>

// Structure to represent a process
struct Process {
    int id;         // Process ID
    int at;         // Arrival Time
    int bt;         // Burst Time
    int ct;         // Completion Time
    int tat;        // Turnaround Time
    int wt;         // Waiting Time
    int type;       // 1 for System Process, 2 for User Process
    int completed;  // Flag to check if process is completed (0 = no, 1 = yes)
};

int main() {
    int n, i;
    int completed_count = 0;
    int current_time = 0;
    float total_wt = 0, total_tat = 0;

    printf("Enter the total number of processes: ");
    scanf("%d", &n);

    struct Process p[n];

    // Input process details
    for (i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("\nProcess %d\n", p[i].id);
        printf("Enter Arrival Time: ");
        scanf("%d", &p[i].at);
        printf("Enter Burst Time: ");
        scanf("%d", &p[i].bt);
        printf("Enter Process Type (1 for System, 2 for User): ");
        scanf("%d", &p[i].type);
        p[i].completed = 0; // Initialize as not completed
    }
```

```
// Scheduling Logic
while (completed_count < n) {
    int selected_process = -1;
    int highest_priority_type = 3; // 1 is highest, 2 is lower
    int earliest_arrival = 999999; // Arbitrary large number

    // Find the appropriate process to execute
    for (i = 0; i < n; i++) {
        if (p[i].completed == 0 && p[i].at <= current_time) {
            // Check priority (System > User)
            if (p[i].type < highest_priority_type) {
                highest_priority_type = p[i].type;
                earliest_arrival = p[i].at;
                selected_process = i;
            }
            // If priority is same, use FCFS (earliest arrival time)
            else if (p[i].type == highest_priority_type && p[i].at < earliest_arrival) {
                earliest_arrival = p[i].at;
                selected_process = i;
            }
        }
    }

    // If a process is found, execute it
    if (selected_process != -1) {
        current_time += p[selected_process].bt;
        p[selected_process].ct = current_time;
        p[selected_process].tat = p[selected_process].ct - p[selected_process].at;
        p[selected_process].wt = p[selected_process].tat - p[selected_process].bt;
        p[selected_process].completed = 1;

        total_tat += p[selected_process].tat;
        total_wt += p[selected_process].wt;

        completed_count++;
    } else {
        // If no process has arrived yet, increment time
        current_time++;
    }
}

// Output the results
printf("\n--- Scheduling Results ---\n");
printf("PID\tType\tAT\tBT\tCT\tTAT\tWT\n");
for (i = 0; i < n; i++) {
    printf("%d\t%s\t%d\t%d\t%d\t%d\t%d\n",
           p[i].id,
           (p[i].type == 1 ? "SYS" : "USR"),
```

```
            p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);
    }

    printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
    printf("Average Waiting Time: %.2f\n", total_wt / n);

    return 0;
}
```

```
PS C:\Users\Aditi Kamath A\Desktop\1BM24CS016> cd "c:\Users\Aditi Kamath A\Desktop\1BM24CS016\"
_fcfs_only.c -o Multilevel_fcfs_only } ; if ($?) { .\Multilevel_fcfs_only }
Enter the total number of processes: 4

Process 1
Enter Arrival Time: 0
Enter Burst Time: 4
Enter Process Type (1 for System, 2 for User): 2

Process 2
Enter Arrival Time: 1
Enter Burst Time: 3
Enter Process Type (1 for System, 2 for User): 1

Process 3
Enter Arrival Time: 2
Enter Burst Time: 2
Enter Process Type (1 for System, 2 for User): 2

Process 4
Enter Arrival Time: 3
Enter Burst Time: 5
Enter Process Type (1 for System, 2 for User): 1

--- Scheduling Results ---
PID     Type    AT      BT      CT      TAT     WT
1       USR     0       4       4       4       0
2       SYS     1       3       7       6       3
3       USR     2       2       14      12      10
4       SYS     3       5       12      9       4

Average Turnaround Time: 7.75
Average Waiting Time: 4.25
```

PROGRAM 3

QUESTION : Write a C program to simulate Real-Time CPU Scheduling algorithms:
Rate- Monotonic
Earliest-deadline First
Proportional scheduling

SOLN : Rate Monotonic Scheduling

```
#include <stdio.h>

int findLCM(int a, int b) {
    int res = (a > b) ? a : b;
    while (1) {
        if (res % a == 0 && res % b == 0) return res;
        res++;
    }
}

int main() {
    int n, i, lcm = 1;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    int burst[n], period[n], rem[n];
    printf("Enter the CPU burst times:\n");
    for (i = 0; i < n; i++) scanf("%d", &burst[i]);

    printf("Enter the time periods:\n");
    for (i = 0; i < n; i++) {
        scanf("%d", &period[i]);
        lcm = (i == 0) ? period[i] : findLCM(lcm, period[i]);
        rem[i] = 0;
    }

    printf("\nLCM = %d\n", lcm);
    printf("Rate Monotone Scheduling:\nPID\tBurst\tPeriod\n");
    for (i = 0; i < n; i++) printf("%d\t%d\t%d\n", i + 1, burst[i], period[i]);

    printf("\nScheduling occurs for %d ms\n", lcm);

    int last_pid = -1;
    for (int t = 0; t < lcm; t++) {

        for (i = 0; i < n; i++) {
            if (t % period[i] == 0) rem[i] = burst[i];
        }

        int selected = -1;
```

```c
        int min_p = 10000;

        for (i = 0; i < n; i++) {
            if (rem[i] > 0 && period[i] < min_p) {
                min_p = period[i];
                selected = i;
            }
        }

        if (selected != last_pid) {
            if (selected == -1) printf("%dms onwards: CPU is idle\n", t);
            else printf("%dms onwards: Process %d running\n", t, selected + 1);
            last_pid = selected;
        }

        if (selected != -1) rem[selected]--;
    }
    return 0;
}
```

```
C:\Users\BMSCECSE-L4>cd C:\Users\BMSCECSE-L4\Desktop\1BM24CS016

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>Rms
Enter the number of processes: 2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100

LCM = 100
Rate Monotone Scheduling:
PID     Burst   Period
1       20      50
2       35      100

Scheduling occurs for 100 ms
0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
70ms onwards: Process 2 running
75ms onwards: CPU is idle

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>
```

SOLN : Earliest Deadline First

```
#include <stdio.h>


int findGCD(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int findLCM(int a, int b) {
    if (a == 0 || b == 0) return 0;
    return (a * b) / findGCD(a, b);
}

int main() {
    int n, i;
    printf("Enter the number of processes: ");
    scanf("%d", &n);

    int burst[n], period[n], deadline[n], rem[n], abs_deadline[n];

    for (i = 0; i < n; i++) {
        printf("Process %d - Burst, Deadline, and Period: ", i + 1);
        scanf("%d %d %d", &burst[i], &deadline[i], &period[i]);
        rem[i] = 0;
        abs_deadline[i] = 0;
    }


    int limit = period[0];
    for (i = 1; i < n; i++) {
        limit = findLCM(limit, period[i]);
    }
printf("\nPID\tBurst\tDeadline\tPeriod\n");
    for (i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t\t%d\n", i + 1, burst[i], deadline[i], period[i]);
    }
    printf("Scheduling occurs for %d ms\n\n", limit);

    for (int t = 0; t < limit; t++) {

        for (i = 0; i < n; i++) {
            if (t % period[i] == 0) {
                rem[i] = burst[i];
                abs_deadline[i] = t + deadline[i];
```

```c
            }
        }

        int selected = -1;
        int min_deadline = 100000;

        for (i = 0; i < n; i++) {
            if (rem[i] > 0) {
                if (abs_deadline[i] < min_deadline) {
                    min_deadline = abs_deadline[i];
                    selected = i;
                }
            }
        }
        if (selected == -1) {
            printf("%dms: CPU is idle.\n", t);
        } else {
            printf("%dms: Task %d is running.\n", t, selected + 1);
            rem[selected]--;
        }
    }

    return 0;
}
```

```
C:\Users\BMSCECSE-L4>cd "C:\Users\BMSCECSE-L4\Desktop\1BM24CS016"

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>Edf
Enter the number of processes: 3
Process 1 - Burst, Deadline, and Period: 3 7 20
Process 2 - Burst, Deadline, and Period: 2 4 5
Process 3 - Burst, Deadline, and Period: 2 8 10

PID     Burst   Deadline        Period
1       3       7               20
2       2       4               5
3       2       8               10
Scheduling occurs for 20 ms

0ms: Task 2 is running.
1ms: Task 2 is running.
2ms: Task 1 is running.
3ms: Task 1 is running.
4ms: Task 1 is running.
5ms: Task 3 is running.
6ms: Task 3 is running.
7ms: Task 2 is running.
8ms: Task 2 is running.
9ms: CPU is idle.
10ms: Task 2 is running.
11ms: Task 2 is running.
12ms: Task 3 is running.
13ms: Task 3 is running.
14ms: CPU is idle.
15ms: Task 2 is running.
16ms: Task 2 is running.
17ms: CPU is idle.
18ms: CPU is idle.
19ms: CPU is idle.
```

SOLN : Proportional Share Scheduling

```
#include<stdio.h>
#include <stdlib.h>
#include <time.h>
typedef struct
{
char name[5]; int tickets; }
Process;
int main() {
int n, total_tickets = 0; float total_T =0.0;
printf("Enter the number of Processes: ");
scanf("%d", &n);
Process p[n];
srand(time(NULL));
for (int i = 0; i < n; i++)
{ printf("\nProcess %d:\n", i + 1);
sprintf(p[i].name, "P%d", i + 1);
printf("Tickets: ");
scanf("%d", &p[i].tickets);
total_tickets += p[i].tickets;
total_T +=p[i].tickets;
}
printf("\n--- Proportional Share Scheduling ---\n");
printf("Enter the Time Period for scheduling: ");
int m;
scanf("%d",&m);
for (int i = 0; i < m; i++)
{
int winning_ticket = rand() % total_tickets + 1;
int accumulated_tickets = 0;
int winner_index;
for (int j = 0; j < n; j++)
{
accumulated_tickets += p[j].tickets;
if (winning_ticket <= accumulated_tickets)
{
winner_index = j; break;
}
}
printf("Tickets picked: %d, Winner: %s\n", winning_ticket,
p[winner_index].name);
}
for (int i = 0; i < n; i++)
{
```

printf("\nThe Process: %s gets %0.2f%% of Processor Time.\n", p[i].name,
((p[i].tickets / total_T) * 100));
} return 0; }

Output:

```
C:\Users\BMSCECSE-L4>cd C:\Users\BMSCECSE-L4\Desktop\1BM24CS016

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>Prop
Enter the number of Processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---
Enter the Time Period for scheduling: 5
Tickets picked: 51, Winner: P3
Tickets picked: 22, Winner: P2
Tickets picked: 17, Winner: P2
Tickets picked: 45, Winner: P3
Tickets picked: 53, Winner: P3

The Process: P1 gets 16.67% of Processor Time.

The Process: P2 gets 33.33% of Processor Time.

The Process: P3 gets 50.00% of Processor Time.

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>
```

PROGRAM 4 :

QUESTION :

4a. Write a C program to simulate producer-consumer problem using semaphores

4b. Write a C program to simulate the concept of Dining Philosophers problem

SOLN : Bounded Buffer

```
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0;
int out = 0;

int empty = BUFFER_SIZE;
int full = 0;

void produce() {
    if (empty == 0) {
        printf("\nBuffer is full! Cannot produce.\n");
        return;
    }

    int item;
    printf("Enter item to produce: ");
    scanf("%d", &item);

    buffer[in] = item;
    printf("Produced: %d at index %d\n", item, in);

    in = (in + 1) % BUFFER_SIZE;
    empty--;
    full++;
}
void consume() {
    if (full == 0) {
        printf("\nBuffer is empty! Cannot consume.\n");
        return;
    }
    int item = buffer[out];
    printf("Consumed: %d from index %d\n", item, out);
    out = (out + 1) % BUFFER_SIZE;
    full--;
    empty++;
```

```
}

int main() {
    int choice;
   printf("Buffer Size: %d\n", BUFFER_SIZE);
while (1) {
        printf("\n1. Produce\n2. Consume\n3. Exit\n");
        printf("Current State: [Full slots: %d | Empty slots: %d]\n", full, empty);
        printf("Enter choice: ");
       if (scanf("%d", &choice) != 1) break;
       switch (choice) {
            case 1:
                produce();
                break;
            case 2:
                consume();
                break;
            case 3:
                printf("Exiting...\n");
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

```
C:\Users\BMSCECSE-L4>cd "C:\Users\BMSCECSE-L4\Desktop\1BM24CS016"

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>PC_Sema
Buffer Size: 5

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 0 | Empty slots: 5]
Enter choice: 1
Enter item to produce: 1
Produced: 1 at index 0

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 1 | Empty slots: 4]
Enter choice: 1
Enter item to produce: 2
Produced: 2 at index 1

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 2 | Empty slots: 3]
Enter choice: 2
Consumed: 1 from index 0

1. Produce
2. Consume
3. Exit
Current State: [Full slots: 1 | Empty slots: 4]
Enter choice: 3
Exiting...

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>
```

SOLN : Dining Philosopher

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#define N 5
pthread_mutex_t forks[N];
pthread_t philosophers[N];
void* philosopher(void* num)
{
int id = *(int*)num;
int left = id;
int right = (id + 1) % N;
while (1)
{
   printf("Philosopher %d is thinking.\n", id);
   sleep(1);
   if (id % 2 == 0)
   {
      pthread_mutex_lock(&forks[left]);
      printf("Philosopher %d picked up left fork %d.\n", id, left);
      pthread_mutex_lock(&forks[right]);
      printf("Philosopher %d picked up right fork %d.\n", id, right);
   }
   else
   {
      pthread_mutex_lock(&forks[right]);
      printf("Philosopher %d picked up right fork %d.\n", id, right);
      fork pthread_mutex_lock(&forks[left]);
      printf("Philosopher %d picked up left fork %d.\n", id, left);
   }
   printf("Philosopher %d is eating.\n", id);
   sleep(2);
   pthread_mutex_unlock(&forks[left]);
   pthread_mutex_unlock(&forks[right]);
   printf("Philosopher %d put down forks %d and %d.\n", id, left,
right);
}
return NULL;
}
int main()
{
   int i;
   int ids[N];
   for (i = 0; i < N; i++)
```

```
    {
        pthread_mutex_init(&forks[i], NULL);
        ids[i] = i;
    }
    for (i = 0; i < N; i++)
    {
        pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);
    }
    for (i = 0; i < N; i++)
    {
        pthread_join(philosophers[i], NULL);
    }
    for (i = 0; i < N; i++)
    {
        pthread_mutex_destroy(&forks[i]);
    }
    return 0;
}
                                                                }
```

```
Philosopher 0 is thinking.
Philosopher 2 is thinking.
Philosopher 1 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 3 picked up right fork 4.
Philosopher 3 picked up left fork 3.
Philosopher 3 is eating.
Philosopher 1 picked up right fork 2.
```

PROGRAM 5

QUESTION : Write a C program to simulate Bankers algorithm and deadlock avoidance

SOLN : Banker algorithm

```c
#include <stdio.h>

int main()
{
    int n, m;
    int i, j, k;


    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int allocation[n][m];
    int max[n][m];
    int need[n][m];
    int available[m];


    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            scanf("%d", &allocation[i][j]);
        }
    }


    printf("\nEnter Max Matrix:\n");
    for(i = 0; i < n; i++)
    {
        for(j = 0; j < m; j++)
        {
            scanf("%d", &max[i][j]);
        }
    }


    printf("\nEnter Available Resources:\n");
```

```
for(i = 0; i < m; i++)
{
   scanf("%d", &available[i]);
}


for(i = 0; i < n; i++)
{
   for(j = 0; j < m; j++)
   {
      need[i][j] = max[i][j] - allocation[i][j];
   }
}

int finish[n];
int safeSequence[n];
int work[m];

for(i = 0; i < n; i++)
{
   finish[i] = 0;
}

for(i = 0; i < m; i++)
{
   work[i] = available[i];
}

int count = 0;

while(count < n)
{
   int found = 0;

   for(i = 0; i < n; i++)
   {
      if(finish[i] == 0)
      {
         for(j = 0; j < m; j++)
         {
            if(need[i][j] > work[j])
               break;
         }

         if(j == m)
         {
            for(k = 0; k < m; k++)
            {
```

```c
                    work[k] += allocation[i][k];
                }

                safeSequence[count++] = i;
                finish[i] = 1;
                found = 1;
            }
        }
    }

    if(found == 0)
    {
        break;
    }
}


printf("\nNeed Matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        printf("%d ", need[i][j]);
    }
    printf("\n");
}


if(count == n)
{
    printf("\nSystem is in SAFE state\n");
    printf("Safe Sequence: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d ", safeSequence[i]);
    }
}
else
{
    printf("\nSystem is NOT in safe state");
}

return 0;
}
```

```
PS C:\Users\Aditi Kamath A\Desktop\1BM24CS016> cd "c:
{ gcc Bankers_for_1.c -o Bankers_for_1 } ; if ($?) {
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2

Enter Max Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3

Enter Available Resources:
3 3 2

Need Matrix:
7 4 3
1 2 2
6 0 0
0 1 1
4 3 1

System is in SAFE state
Safe Sequence: P1 P3 P4 P0 P2
PS C:\Users\Aditi Kamath A\Desktop\1BM24CS016>
```

SOLN : Deadlock avoidance algorithm

```
#include <stdio.h>

int main()
{
    int n, m;
    int i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int allocation[n][m];
    int request[n][m];
    int available[m];

    // Input Allocation Matrix
    printf("\nEnter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
    {
```

```
    for(j = 0; j < m; j++)
    {
        scanf("%d", &allocation[i][j]);
    }
}

// Input Request Matrix
printf("\nEnter Request Matrix:\n");
for(i = 0; i < n; i++)
{
    for(j = 0; j < m; j++)
    {
        scanf("%d", &request[i][j]);
    }
}

// Input Available Resources
printf("\nEnter Available Resources:\n");
for(i = 0; i < m; i++)
{
    scanf("%d", &available[i]);
}

int finish[n];
int work[m];

// Initialize Work = Available
for(i = 0; i < m; i++)
{
    work[i] = available[i];
}

// Initialize Finish[]
for(i = 0; i < n; i++)
{
    int flag = 0;

    for(j = 0; j < m; j++)
    {
        if(allocation[i][j] != 0)
        {
            flag = 1;
            break;
        }
    }

    if(flag == 1)
```

```
        finish[i] = 0;
      else
        finish[i] = 1;
    }

    // Deadlock Detection Algorithm
    for(k = 0; k < n; k++)
    {
      for(i = 0; i < n; i++)
      {
        if(finish[i] == 0)
        {
          for(j = 0; j < m; j++)
          {
            if(request[i][j] > work[j])
              break;
          }

          if(j == m)
          {
            for(j = 0; j < m; j++)
            {
              work[j] += allocation[i][j];
            }
            finish[i] = 1;
          }
        }
      }
    }
    int deadlock = 0;
    printf("\n");

    for(i = 0; i < n; i++)
    {
      if(finish[i] == 0)
      {
        printf("Process P%d is deadlocked\n", i);
        deadlock = 1;
      }
    }
    if(deadlock == 0)
    {
      printf("No Deadlock Detected\n");
    }

    return 0;
}
```

```
PS C:\Users\Aditi Kamath A\Desktop\1BM24CS016> cd "c:\User
ltiple.c -o Bankers_multiple } ; if ($?) { .\Bankers_multi
Enter number of processes: 5
Enter number of resources: 3

Enter Allocation Matrix:
0 1 0
2 0 0
3 0 3
2 1 1
0 0 2

Enter Request Matrix:
0 0 0
2 0 2
0 0 0
1 0 0
0 0 2

Enter Available Resources:
0 0 0

No Deadlock Detected
PS C:\Users\Aditi Kamath A\Desktop\1BM24CS016>
```

PROGRAM 6 :

QUESTION : Write a C program to simulate Memory Allocation (First Fit, Best Fit, Worst Fit)

SOLN : Memory Allocation

```c
#include <stdio.h>

#define MAX 100

void printAllocation(int processSize[], int n, int allocation[]) {
    printf("\nProcess No.\tProcess Size\tBlock No.\n");
    for (int i = 0; i < n; i++) {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);
        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
    printf("\n");
}
void firstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[MAX];
    int is_allocated[MAX] = {0};
    for (int i = 0; i < n; i++) {
        allocation[i] = -1;
    }
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i] && !is_allocated[j]) {
                allocation[i] = j;
                is_allocated[j] = 1;
                break;
            }
        }
    }
    printf("\n FIRST FIT ALLOCATION ");
    printAllocation(processSize, n, allocation);
}
void bestFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[MAX];
    int is_allocated[MAX] = {0};

    for (int i = 0; i < n; i++) {
        allocation[i] = -1;
```

```
    }
    for (int i = 0; i < n; i++) {
        int bestIdx = -1;
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i] && !is_allocated[j]) {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx]) {
                    bestIdx = j;
                }
            }
        }
        if (bestIdx != -1) {
            allocation[i] = bestIdx;
            is_allocated[bestIdx] = 1;
        }
    }
    printf("\nBEST FIT ALLOCATION");
    printAllocation(processSize, n, allocation);
}
void worstFit(int blockSize[], int m, int processSize[], int n) {
    int allocation[MAX];
    int is_allocated[MAX] = {0};
    for (int i = 0; i < n; i++) {
        allocation[i] = -1;
    }
    for (int i = 0; i < n; i++) {
        int worstIdx = -1;
        for (int j = 0; j < m; j++) {
            if (blockSize[j] >= processSize[i] && !is_allocated[j]) {
                if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx]) {
                    worstIdx = j;
                }
            }
        }
        if (worstIdx != -1) {
            allocation[i] = worstIdx;
            is_allocated[worstIdx] = 1;
        }
    }
    printf("\nWORST FIT ALLOCATION ");
    printAllocation(processSize, n, allocation);
}
int main() {
    int blockSize[MAX], processSize[MAX];
    int m, n;
    printf("Enter the number of memory blocks: ");
```

```c
    scanf("%d", &m);
    printf("Enter the size of each memory block:\n");
    for (int i = 0; i < m; i++) {
        printf("Block %d: ", i + 1);
        scanf("%d", &blockSize[i]);
    }
    printf("\nEnter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the size of each process:\n");
    for (int i = 0; i < n; i++) {
        printf("Process %d: ", i + 1);
        scanf("%d", &processSize[i]);
    }
    firstFit(blockSize, m, processSize, n);
    bestFit(blockSize, m, processSize, n);
    worstFit(blockSize, m, processSize, n);
    return 0;
}
```

```
PS C:\Users\Aditi Kamath A\Desktop\1BM24CS016> cd "c:\Users\Aditi Kamath A\Desktop\1BM24CS016\" ; if
Enter the number of memory blocks: 5
Enter the size of each memory block:
Block 1: 100
Block 2: 500
Block 3: 200
Block 4: 300
Block 5: 600

Enter the number of processes: 4
Enter the size of each process:
Process 1: 212
Process 2: 417
Process 3: 112
Process 4: 426

 FIRST FIT ALLOCATION
Process No.     Process Size    Block No.
1               212             2
2               417             5
3               112             3
4               426             Not Allocated


BEST FIT ALLOCATION
Process No.     Process Size    Block No.
1               212             4
2               417             2
3               112             3
4               426             5
```

```
WORST FIT ALLOCATION
Process No.     Process Size    Block No.
1               212             5
2               417             2
3               112             4
4               426             Not Allocated

PS C:\Users\Aditi Kamath A\Desktop\1BM24CS016>
```

PROGRAM 7 :

QUESTION : Write a C program to simulate Page Replacement

```
#include <stdio.h>
#include <stdbool.h>

#define MAX_PAGES 100


void printFrames(int frames[], int num_frames) {
    printf("[");
    int count = 0;
    for (int i = 0; i < num_frames; i++) {
        if (frames[i] != -1) {
            printf("%d", frames[i]);
            count++;
        }
    }
    printf("]\n");
}


void fifoPageReplacement(int pages[], int num_pages, int num_frames) {
    printf("\n--- FIFO Page Replacement ---\n");

    int frames[num_frames];
    for (int i = 0; i < num_frames; i++) frames[i] = -1;

    int page_faults = 0;
    int next = 0;

    for (int i = 0; i < num_pages; i++) {
        int page = pages[i];
        bool is_present = false;


        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == page) {
                is_present = true;
                break;
            }
        }

        printf("Page %d -> ", page);

        if (!is_present) {

            frames[next] = page;
            next = (next + 1) % num_frames;
            page_faults++;
```

```c
        }

        printFrames(frames, num_frames);
    }
    printf("Total Page Faults (FIFO): %d\n", page_faults);
}


void lruPageReplacement(int pages[], int num_pages, int num_frames) {
    printf("\n--- LRU Page Replacement ---\n");

    int frames[num_frames];
    int last_used[num_frames];
    for (int i = 0; i < num_frames; i++) {
        frames[i] = -1;
        last_used[i] = -1;
    }

    int page_faults = 0;

    for (int i = 0; i < num_pages; i++) {
        int page = pages[i];
        bool is_present = false;
        int present_index = -1;


        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == page) {
                is_present = true;
                present_index = j;
                break;
            }
        }

        printf("Page %d -> ", page);

        if (is_present) {

            last_used[present_index] = i;
        } else {

            page_faults++;


            int empty_index = -1;
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) {
                    empty_index = j;
                    break;
                }
```

```
            }

            if (empty_index != -1) {

                frames[empty_index] = page;
                last_used[empty_index] = i;
            } else {

                int lru_index = 0;
                int min_time = last_used[0];
                for (int j = 1; j < num_frames; j++) {
                    if (last_used[j] < min_time) {
                        min_time = last_used[j];
                        lru_index = j;
                    }
                }
                frames[lru_index] = page;
                last_used[lru_index] = i;
            }
        }

        printFrames(frames, num_frames);
    }
    printf("Total Page Faults (LRU): %d\n", page_faults);
}


void optimalPageReplacement(int pages[], int num_pages, int num_frames) {
    printf("\n--- Optimal Page Replacement ---\n");

    int frames[num_frames];
    for (int i = 0; i < num_frames; i++) frames[i] = -1;

    int page_faults = 0;

    for (int i = 0; i < num_pages; i++) {
        int page = pages[i];
        bool is_present = false;

        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == page) {
                is_present = true;
                break;
            }
        }

        printf("Page %d -> ", page);

        if (!is_present) {
            page_faults++;
```

```
            int empty_index = -1;
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) {
                    empty_index = j;
                    break;
                }
            }

            if (empty_index != -1) {
                frames[empty_index] = page;
            } else {

                int replace_index = -1;
                int farthest_use = -1;

                for (int j = 0; j < num_frames; j++) {
                    int next_use = -1;

                    for (int k = i + 1; k < num_pages; k++) {
                        if (pages[k] == frames[j]) {
                            next_use = k;
                            break;
                        }
                    }


                    if (next_use == -1) {
                        replace_index = j;
                        break;
                    }


                    if (next_use > farthest_use) {
                        farthest_use = next_use;
                        replace_index = j;
                    }
                }

                frames[replace_index] = page;
            }
        }

        printFrames(frames, num_frames);
    }
    printf("Total Page Faults (Optimal): %d\n", page_faults);
}

int main() {
    int num_pages, num_frames;
```

```c
    int pages[MAX_PAGES];


    printf("Enter number of pages: ");
    if (scanf("%d", &num_pages) != 1 || num_pages <= 0 || num_pages > MAX_PAGES) {
        printf("Invalid number of pages.\n");
        return 1;
    }

    printf("Enter page reference string:\n");
    for (int i = 0; i < num_pages; i++) {
        scanf("%d", &pages[i]);
    }

    printf("Enter number of frames: ");
    if (scanf("%d", &num_frames) != 1 || num_frames <= 0) {
        printf("Invalid number of frames.\n");
        return 1;
    }
    fifoPageReplacement(pages, num_pages, num_frames);
    lruPageReplacement(pages, num_pages, num_frames);
    optimalPageReplacement(pages, num_pages, num_frames);

    return 0;
}
```

```
C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>PageRep
Enter number of pages: 21
Enter page reference string:
5 0 1 0 2 3 0 2 4 3 3 2  0 2 1 2 7 0 1 1 0
Enter number of frames: 3

--- FIFO Page Replacement ---
Page 5 -> [5]
Page 0 -> [50]
Page 1 -> [501]
Page 0 -> [501]
Page 2 -> [201]
Page 3 -> [231]
Page 0 -> [230]
Page 2 -> [230]
Page 4 -> [430]
Page 3 -> [430]
Page 3 -> [430]
Page 2 -> [420]
Page 0 -> [420]
Page 2 -> [420]
Page 1 -> [421]
Page 2 -> [421]
Page 7 -> [721]
Page 0 -> [701]
Page 1 -> [701]
Page 1 -> [701]
Page 0 -> [701]
Total Page Faults (FIFO): 11

--- LRU Page Replacement ---
Page 5 -> [5]
Page 0 -> [50]
Page 1 -> [501]
Page 0 -> [501]
Page 2 -> [201]
Page 3 -> [203]
Page 0 -> [203]
Page 2 -> [203]
Page 4 -> [204]
Page 3 -> [234]
Page 3 -> [234]
Page 2 -> [234]
Page 0 -> [230]
Page 2 -> [230]
Page 1 -> [210]
Page 2 -> [210]
Page 7 -> [217]
Page 0 -> [207]
Page 1 -> [107]
Page 1 -> [107]
Page 0 -> [107]
Total Page Faults (LRU): 12
```

```
--- Optimal Page Replacement ---
Page 5 -> [5]
Page 0 -> [50]
Page 1 -> [501]
Page 0 -> [501]
Page 2 -> [201]
Page 3 -> [203]
Page 0 -> [203]
Page 2 -> [203]
Page 4 -> [243]
Page 3 -> [243]
Page 3 -> [243]
Page 2 -> [243]
Page 0 -> [203]
Page 2 -> [203]
Page 1 -> [201]
Page 2 -> [201]
Page 7 -> [701]
Page 0 -> [701]
Page 1 -> [701]
Page 1 -> [701]
Page 0 -> [701]
Total Page Faults (Optimal): 9

C:\Users\BMSCECSE-L4\Desktop\1BM24CS016>
```